Laboratorio di Algoritmi e strutture dati

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso **2**

2 Argomenti del corso **2**

3 Gestione del corso (prove intercorso) **2**

4 Esame **3**

5 Lezione del 11-03 **3**

5.1 Struttura modulare di un programma 3

5.1.1 Struttura di un progetto 4

6 Lezione del 12-03 **5**

6.1 Tipi fondamentali, Puntatori e Riferimenti 5

6.1.1 Dichiarazione di variabili 5

6.1.2 Tipi fondamentali in c++ 5

6.1.3 Tipi const 6

6.1.4 Puntatori 6

6.1.5 Void pointer 7

6.1.6 Riferimenti 7

6.1.7 Lvalue & Rvalue 9

7 Lezione del 18-03 **9**

7.1 Allocazione dinamica della memoria 9

7.1.1 Esempio per la locazione 11

7.2 Tipi definiti dall'utente 12

7.3 Le enumeration 13

7.4 Libreria iostream & funzioni di `I/O` 14

7.4.1 Classe istream 14

7.4.2 Classi ostream 14

1 Introduzione al corso

`Ricevimento` → Lunedì ore 16-18 (Teams).

`Email` → fabio.mogavero@unina.it

Link al download del libro di Data Structure & Algorithm Analysis → clicca per il download

2 Argomenti del corso

- Elementi di Linguaggio C++
- Tipi di dato astratto e relative implementazioni in C++
- Progettazione di una libreria contenitore di dati
- Strutture dati elementari (vettori, liste, pile, code)
- Alberi binari di ricerca e iteratori sui dati
- Matrici e grafi

3 Gestione del corso (prove intercorso)

- Esercizio Vettori e liste (26 Marzo - 11 Aprile);
- Esercizio Pile e Code (9 Aprile - 25 Aprile);
- Esercizio Alberi Binari di Ricerca (23 Aprile - 9 Maggio);
- Esercizio Iteratori su Alberi (7 Maggio - 23 Maggio);
- Esercizio su Matrici (21 Maggio - 6 Giugno);
- Esercizio sui Grafi (4 Giugno - 20 Giugno).

Per la consegna:

- Invio file .zip o tar.gz alla mail del prof con oggetto "Consegna Esercizio x (Cognome-Nome-Matricola)"
- Formato File: Cognome-Nome-Matricola(Esercizio x).zip *o* Cognome-Nome-Matricola.tar.gz
- Scadenza improrogabili → ore 23:59 dell'ultimo giorno previsto per l'esercizio

4 Esame

Esame in presenza:

- Prova al calcolatore + discussione orale

Per chi consegna tutti gli elaborati si ha la possibilità di accedere all'esame semplificato (reimplementazioni, implementazione di una `vecchia/nuova` funzionalità di una libreria tra quelle implementate)

Per chi non consegna tutti gli elaborati si è obbligati ad accedere all'esame completo (implementazione da zero di una libreria)

Prerogative per entrambe le modalità è l'obbligo della consegna di tutte le librerie con data ultima precedente alla data di esame a cui si vuole accedere

Nel caso di modalità telematica nel caso dell'esame semplificato si svolgerà solo la parte orale

5 Lezione del 11-03

5.1 Struttura modulare di un programma

Primo esempio (main.cpp)

```
#include <iostream> // libreria standard c++ I/O
// dichiarazione dello spazio dei tipi
//(accesso alle funzionalità senza la specificazione "std::")
using namespace std;
int main(){
    cout << "Hello World!" << endl;
}
```

Di seguito invece il `build.sh` (script bash)

```
#!/bin/bash
g++ [-03] -o main main.cpp
```

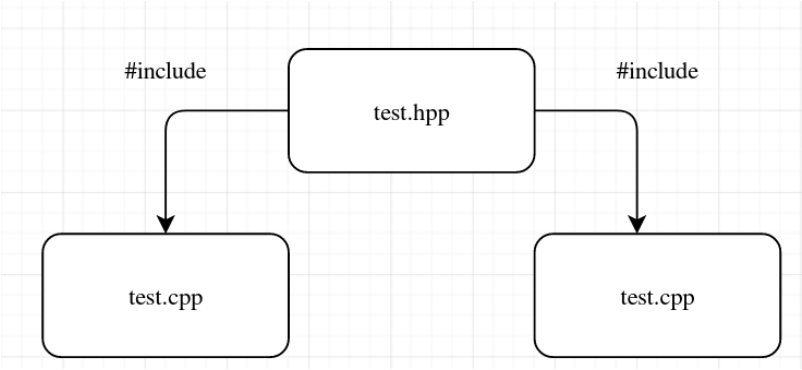
Dove:

- g++ → compilatore GNU;
- `[-O3]` → parametri di ottimizzazione opzionale;
- -o main → `output/file eseguibile`;
- main.cpp → codice sorgente

Nel momento in cui ci siano più file da compilare vanno aggiunti manualmente (l'ordine è indefferente)

5.1.1 Struttura di un progetto

Nel caso di più file si avrà un file principale più altri che eseguiranno effettivamente le operazioni:



test.hpp → file header. Si presuppone che abbia delle macro iniziali e finali che viene letto dal precompilatore

```
// main.cpp
#include <iostream>
#include "test.hpp"
using namespace std;
int main(){
    cout << "Chiamata a funzione";
    test();
}
```

```cpp
// test.hpp
// Le macro vengono risolge dal preprocessore
#ifndef TEST_HPP
#define TEST_HPP
void test();
#endif

// test.cpp
#include <iostream>
#include "test.hpp"
void test(){
    std::cout<<"OK!"<< endl;
}
```

6 Lezione del 12-03

6.1 Tipi fondamentali, Puntatori e Riferimenti

6.1.1 Dichiarazione di variabili

`type varname [= defaul value]`

`type varname [numero elementi] [{` $\text{val}_0, \ldots, \text{val}_n$ `}]`

6.1.2 Tipi fondamentali in c++

- Bool (`true/false`);
- `[unsigned]` char;
- `[unsigned]` short;
- `[unsigned]` int;
- `[unsigned]` long int;
- `[unsigned]` long long int;
- float;
- double;
- long double

In alcuni compilatori è possibile delle abbrevazioni per tipi unsigned int e long del tipo:

- uint;
- ulong;

6.1.3 Tipi const

```
const type varname = value;
```

Ritornano utili anche nel passaggio dei parametri nelle chiamate a funzione

6.1.4 Puntatori

```
type* varname [=default address]
```

es.

```
#include <iostream>
using namespace std;
int main(){
    char c = 'a';
    char* p = &c;
    cout << "Il valore di *p è "<< *p << endl;
    p= nullptr; //corrisponde alla macro NULL del linguaggio C
    cout << p << endl;
}
```

```
Il valore di *p è a
```

```
#include <iostream>
using namespace std;
int main(){
    const char c = 'a';
    char* p = &c; // ERRORE
    const char* p = &c; // CORRETTO
    cout << *p << endl;
}
```

6.1.5 Void pointer

```
void* varname [= ...]
```

Un void pointer è un puntatore che non è associato a nessun tipo specifico. Puo mantenere un indirizzo di ogni tipo e può essere typecasted a ogni tipo.

Indefinire un puntatore in cui non è noto a priori il tipo di dato a cui si vuole puntare

```
#include <iostream>
using namespace std;
int main(){
    char c = 'a';
    void* p = &c;
    //cout << *p << endl; // ERRORE
    cout << *((char*)p) << endl; // CORRETTO
    cout << *(static_cast<char*>(p)) << endl; // MODO MIGLIORE
}
```

6.1.6 Riferimenti

```
type & varname = ...
```

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint & e = d;
    e++;
    cout << "Il valore di d è " << d;
}
// Stamperà il valore 8
```

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint e = d;
    e++;
    cout << "Il valore di d è " << d;
}
// Stamperà il valore 7
```

Con i puntatori invece:

```
#include <iostream>
using namespace std;
int main(){
    uint d = 7;
    uint* e = &d;
    (*e)++;
    cout << "Il valore di d è " << d;
}
```

```
Il valore di d è 8
```

```
int & i = 1; // ERRORE
const int & i = 1; // CORRETTO
```

6.1.7 Lvalue & Rvalue

Differenza tra Lvalue e Rvalue

In C++ un Lvalue è qualcosa che punta a una specifica locazione di memoria. Un Rvalue invece è qualcosa che non punta da nessuna parte.

In generale gli Rvalues sono temporanei e di vita breve, mentre gli Lvalues hanno una vita lunga tanto quanto quella della variabile.

```
#include <iostream>
using namespace std;
string f(){return string ("Ciao");};
int main(){
    string var = f();
    string stringa1 = move(f());
    string && stringa2 = f(); // Riferimento Rvalue
    string stringa3 = move(stringa2); // Spostiamo il valore di stringa2 in stringa3 senza copiarla
    cout << var<< endl;
    cout << stringa1<< endl;
    cout << stringa2<< endl;
    cout << stringa3;
}
```

Ciao
Ciao
Ciao

7 Lezione del 18-03

7.1 Allocazione dinamica della memoria

Dal linguaggio C:

- new → malloc();
- delete → free();
- realloc non ha diretta corrispondenza in C++
- New solleva un'eccezione;
- New non restituisce *nullptr* quando fallisce ("bad-alloc");

- L'uso di `new/delete` non va combinato con chiamate a funzioni `malloc/realloc/free`;
- No naked "new" "delete".
- Non considerano gli smart-pointer;

```
uint num = espressione;
type* var = new type[num]; //dimensione array
delete[] var;

// Allocazione di tipo unico
// Tra le parentesi ul potenziale del valore di default
ObjectType* var = new ObjectType();
...
delete var;
```

7.1.1 Esempio per la locazione

```
#include <iostream>
using namespace std;
int main(){
try {
    // ulong* ulptr = new ulong;  Non inizializzato
    ulong* ulptr = new ulong(5); // Inizializzato
    cout << (*ulptr)++ << endl;
    cout << *ulptr << endl;
    delete ulptr;
    //delete ulptr;
}catch (bad_alloc exc){
        cout << "Quite rare exception !" <<endl;
    }
try{
const ulong arraysize = 100000000000000; // Ottengo il cout della catch
ulong* ulptr = new ulong[arraysize];

cout << ulptr[0]++ << endl;
cout << ulptr[1] << endl;
cout << ulptr[2]-- << endl;
cout << ulptr[0] << endl;
cout << ulptr[1] << endl;
cout << ulptr[2] << endl;
delete[] ulptr;

}catch(bad_alloc exc){
        cout << "Quite rare exception !" <<endl;
    }
}
```

Nel caso in cui si chiami una delete due volte di seguito si andrà in contro a `free(): double free detected in tcache 2`. Per evitar ciò un workaround è quello di settare a la variabile a *nullptr*

7.2 Tipi definiti dall'utente

Le struct sono classi con accesso pubblico agli attributi e alle funzioni membro; nessuna altra restrizione.

struct strudente{ campi };

Un esempio di struct:

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

struct Studente{
    string Nome;
    string Cognome;
    string Matricola;
    uint id;
};
int main(){
}
```

Member access → StructNomeVal.Campo.

Accesso via pointer → poitnerVarName -> campo (equivalente ad (*PointerVarNome).Campo)

```
#include <iostream>
using namespace std;
struct Studente{
    uint Id;
    string Matricola = "N86000";
    string Nome = "Alan";
    string Cognome = "Turing";
    // Creo un costruttore con 0 o tutti gli argomenti
    Studente() = default;
    Studente(ulong id, string matr, string nome, string cognome){
        Id = id;
        Matricola = matr;
        Nome = nome;
        Cognome = cognome;
```

```
    }
};
int main(){
    Studente stu;
    cout << "Nome: " << stu.Nome <<endl;

    // Crea un costruttore che rimpiazza i valori definiti
    Studente stud1{2, "N860001", "Gennaro", "Esposito"};
    cout << "Nome: " << stud1.Nome << endl;
    // Necessita del costruttore sopra definito
    Studente stud2(3, "N860002", "Alonzo", "Church");
    cout << "Nome: " << stud2.Nome << endl;
}
```

7.3 Le enumeration

```
enum class Nome{elemento, ..., elementoN}; // In C++
enum Nome {...};
enum class Color{white, gray, black};
enum class Color3{white, gray, black};
enum class Color4{white, gray, black};
Colore colore = Color::white;
Color4 colore4 = Color4::white; // Corretto
Color4 colore4 = Color::white; // ERRATO!
```

In C non è possibile definire due enum con campi uguali (gli elementi sono visibili a livello globale). Gli enum possono essere confrontati e assegnati. Le variabili di tipo enumerativo possono ovviamente avere un valore iniziale di default

```
#include <iostream>
using namespace std;
enum class Colore {Bianco, Grigio, Rosso}; // C++
enum class Colore1 {Grigio, Rosso}; // C++
enum Colore2{Rosso, Giallo};
enum Colore3{Blu, Verde};
int main(){
    Colore color = Colore::Grigio;
    cout << (color < Colore::Bianco) << endl;
    cout << (color == Colore::Grigio) << endl;
}
```

Il confronto tra due enumeration diverse non è possibile (non è definito)

7.4 Libreria iostream & funzioni di `I/O`

Operatori principali » (get-from) e « (put-to). Standard stream:

- cout, cerr, clog;
- cin.

7.4.1 Classe istream

Analisi dello stato:

- good();
- eaf();
- fail();
- bad();
- get(c) → (char);
- getline(p,n) → (char + numero di caratteri da leggere);

7.4.2 Classi ostream

- put(c);
- write(p,n) → (char + numero di caratteri da scrivere);

```
#include <iostream>
using namespace std;
struct Studente{
    uint Id;
    string Matricola = "N86000";
    string Nome = "Alan";
    string Cognome = "Turing";
    friend ostream& operator<<(ostream& outstr, const Studente& stu);
    friend istream& operator>>(istream& instr,Studente& stu);
    private:
       ulong SecureNum = 0x232523;
};
```

```
// Per la stampa dei dati
ostream& operator<<(ostream& outstr, const Studente& stu){
    outstr << "Id: " << stu.Id << " Matricola: "
           << stu.Matricola << " Nome: "
           << stu.Nome << " Cognome:" << stu.Cognome;
}
// Per l'inserimento dei dati
istream& operator>>(istream& instr,Studente& stu){
    instr >> stu.Id >>  stu.Matricola >> stu.Nome >> stu.Cognome;
    instr >> stu.SecureNum;
}
int main(){
Studente stud;
    cout << stud << endl;
}
```